VetCare - Guía de estudio del código

Qué es cada archivo y cada instrucción, por qué se usa y qué función cumple. Corta y para repasar antes de exponer.

1. La estructura: ¿por qué en capas?

Las 4 carpetas del proyecto

Cada carpeta es un proyecto separado con una sola responsabilidad. La regla: **Api -> Negocio -> Datos**. Nadie llama hacia arriba y los controladores nunca tocan la base de datos.

Proyecto	Qué contiene	Pregunta que responde
VetCare.Datos	Entidades (tablas), ContextoVetCare, Repositorios	¿Cómo guardo y leo?
VetCare.Negocio	Servicios, DTOs, Excepciones, reglas	¿Qué se permite hacer?
VetCare.Api	Controladores, JWT, Middleware, Swagger, wwwroot	¿Cómo se pide y se responde?
VetCare.Pruebas	Pruebas unitarias con Moq	¿Funciona la lógica?

2. Interfaces: ¿por qué algunos archivos empiezan con I?

IRepositorio.cs, IHasheador.cs, IServicioAutenticacion.cs, IServicioToken.cs...

La **I** significa **Interface**. Una interfaz es un **contrato**: dice qué métodos existen, pero no cómo funcionan. La clase que la implementa (sin la I) es la que hace el trabajo. Se usa porque: (1) los controladores y servicios dependen del contrato y no de una clase concreta; (2) en las pruebas se reemplaza por un mock (Moq); (3) se puede cambiar la implementación sin tocar el resto.

```
public interface IHasheador {                 // contrato
    string HashearContraseña(string contraseña);
    bool VerificarContraseña(string c, string hash);
}
public class HasheadorBcrypt : IHasheador { ... } // quien hace el trabajo
```

Inyección de dependencias (AddScoped)

En Program.cs se registra qué clase usar para cada interfaz. ASP.NET la entrega sola en el constructor. **AddScoped** = una instancia por petición HTTP (necesario para compartir el mismo DbContext).

```
builder.Services.AddScoped<IServicioCliente, ServicioCliente>();
// el controlador pide IServicioCliente y recibe un ServicioCliente
```

3. Capa de datos (VetCare.Datos)

Entidades (carpeta Entidades)

Cada clase es una tabla; cada propiedad, una columna. Ejemplo: Cliente.cs -> tabla clientes. Las propiedades como List<Mascota> son las relaciones (un cliente tiene muchas mascotas).

ContextoVetCare.cs (el DbContext)

Es el puente entre C# y PostgreSQL (Entity Framework Core, el ORM). Guarda los DbSet y configura el modelo.

```
public DbSet<Cliente> Clientes => Set<Cliente>();
// cada DbSet = una tabla. Clientes.Add(x) + SaveChanges() = INSERT
```

public, private, static, const, readonly

public: visible desde otras clases. **private**: solo dentro de la clase. **static**: se usa sin crear objeto (FechasClinica.InicioDeHoy()). **const**: valor fijo (NombresRoles.Administrador). **readonly**: se asigna una vez en el constructor (_clientes).

Constructor (dos cosas distintas con ese nombre)

1) Constructor de clase: método con el nombre de la clase que se ejecuta al crear el objeto; aquí recibe las dependencias inyectadas. **2) La variable 'constructor'** dentro de OnModelCreating o Program.cs es solo un nombre de variable: en el contexto es el ModelBuilder (dibuja el modelo) y en Program.cs es el WebApplicationBuilder.

```
public ServicioCliente(IRepositorioCliente clientes) { _clientes = clientes; }  // 1
protected override void OnModelCreating(ModelBuilder constructor) { ... }        // 2
```

HasOne, WithOne, WithMany, HasForeignKey, HasIndex, IsUnique, HasPrecision

Son instrucciones de EF Core (API fluida) para describir relaciones y reglas. **HasOne/WithOne**: relación uno a uno. **HasForeignKey**: cuál columna guarda el id del otro. **HasIndex().IsUnique()**: no se repite (correo, DPI). **HasPrecision(10,2)**: decimal con 2 decimales.

```
constructor.Entity<RegistroFallecimiento>()
    .HasOne(r => r.Mascota)            // un registro tiene UNA mascota
    .WithOne(m => m.RegistroFallecimiento)  // y una mascota tiene UN registro
    .HasForeignKey<RegistroFallecimiento>(r => r.MascotaId);
```

foreach

Recorre una colección elemento por elemento. En ContextoVetCare se usa dos veces: para cambiar el borrado en todas las llaves foráneas y para renombrar todas las tablas y columnas.

```
foreach (var llave in constructor.Model.GetEntityTypes().SelectMany(e => e.GetForeignKeys()))
{
    llave.DeleteBehavior = DeleteBehavior.Restrict;
}
```

DeleteBehavior.Restrict

Define qué pasa al borrar un registro padre. **Restrict** = no se permite si tiene hijos (error). Así borrar un cliente no borra en cascada sus mascotas y su historial. La regla de negocio decide qué se puede eliminar.

El bloque de nombres: minúscula con guiones bajos (el que no entendías)

EF Core crea por defecto tablas como **Clientes** y columnas como **FechaNacimiento**. Este bloque las cambia a **clientes** y **fecha_nacimiento**, que es el estilo normal de PostgreSQL. Paso a paso: (1) **foreach entidad** recorre todas las tablas del modelo. (2) **GetTableName / SetTableName** lee el nombre y lo reemplaza. (3) El **foreach propiedad** interno hace lo mismo con cada columna. (4) **AMinusculaConGuiones** es el método que convierte el texto.

```
foreach (var entidad in constructor.Model.GetEntityTypes())      // cada tabla
{
    var nombreTabla = entidad.GetTableName();                     // 'RegistrosVacunacion'
    if (nombreTabla != null)
        entidad.SetTableName(AMinusculaConGuiones(nombreTabla));  // 'registros_vacunacion'
    foreach (var propiedad in entidad.GetProperties())            // cada columna
        propiedad.SetColumnName(AMinusculaConGuiones(propiedad.Name)); // 'fecha_nacimiento'
}
```

Regex: Regex.Replace(texto, "(?<!^)([A-Z])", "_$1")

Regex = expresión regular, un patrón para buscar texto. **[A-Z]** busca una letra mayúscula. **(?<!^)** significa 'que NO esté al inicio' (para no poner guion antes de la primera letra). **([A-Z])** la captura y **_$1** la reemplaza por un guion bajo más la misma letra. Después **ToLowerInvariant()** pasa todo a minúscula.

```
'FechaNacimiento' -> 'Fecha_Nacimiento' -> 'fecha_nacimiento'
'PesoKg'          -> 'Peso_Kg'          -> 'peso_kg'
```

Dictionary (diccionario)

Colección de pares **clave -> valor**, como un índice. Se busca por la clave al instante. En SembradorDatos: especie -> lista de razas; y razas["Perro|Poodle"] devuelve el id de esa raza.

```
var catalogo = new Dictionary<string, string[]> {
    ["Perro"] = new[] { "Poodle", "Bulldog" }, ["Gato"] = new[] { "Siames" } };
foreach (var (especie, razas) in catalogo) { ... }  // recorre clave y valor
```

DateTime.UtcNow

Fecha y hora actual en **UTC** (hora universal, sin zona). PostgreSQL guarda las fechas como timestamptz y solo acepta UTC, y así el dato no depende de la computadora. Guatemala es UTC-6, por eso FechasClinica.InicioDeHoy() resta 6 horas.

```
public DateTime FechaCreacion { get; set; } = DateTime.UtcNow;
```

Repositorios (RepositorioBase<T> y los específicos)

Un repositorio es la clase que habla con la base de datos para una entidad. RepositorioBase<T> es **genérico**: una sola implementación de crear, listar, actualizar y eliminar para cualquier entidad. Los específicos (RepositorioCliente) agregan consultas propias. Todo con LINQ, sin SQL manual (el ORM lo traduce).

```
return await _contexto.Clientes.AsNoTracking()
    .Where(c => c.Nombre.ToLower().Contains(termino)).ToListAsync();
```

4. Capa de negocio (VetCare.Negocio)

Servicios (ServicioCliente, ServicioCita, ServicioFactura...)

Aquí viven las reglas: no repetir DPI, no vacunar mascotas fallecidas, no cruzar citas, descontar stock. Si una regla falla, lanzan una excepción de negocio. No saben nada de HTTP ni de SQL.

Excepciones de negocio

ExcepcionNoEncontrada (404), ExcepcionConflicto (409), ExcepcionReglaNegocio (422), ExcepcionNoAutenticado (401). El servicio solo avisa 'qué pasó'; el middleware lo traduce al código HTTP.

DTOs (carpeta Dtos)

Data Transfer Object: clase solo para entrada y salida de la API. Evita exponer las entidades (por ejemplo, el hash de la contraseña) y lleva las validaciones ([Required], [Range], [EmailAddress]).

ConversorPeso.cs

Clase estática que convierte kg <-> lb (1 kg = 2.2046226218 lb), redondea a 2 decimales y valida el rango 0.01-200 kg. La base guarda siempre kilogramos.

async / await / Task

Los métodos que van a la base de datos son asíncronos: await espera la respuesta sin bloquear el servidor. Task es la 'promesa' del resultado.

5. Capa de presentación (VetCare.Api)

Controladores

Reciben la petición HTTP y llaman al servicio; no tienen reglas. Atributos clave: [ApiController], [Route("api/clientes")], [HttpGet]/[HttpPost], [FromBody] (datos del JSON), [Authorize(Roles = ...)] (quién puede entrar), [ProducesResponseType] (documenta códigos en Swagger).

HasheadorBcrypt.cs

Implementa IHasheador con BCrypt. **HashearContraseña** convierte la contraseña en un hash que no se puede revertir (se guarda este, nunca la contraseña). **VerificarContraseña** compara lo que escribe el usuario con el hash.

```
BCrypt.Net.BCrypt.HashPassword(contraseña);
BCrypt.Net.BCrypt.Verify(contraseña, hash);
```

OpcionesJwt.cs

Clase de configuración. Lee la sección "Jwt" de appsettings.json: Clave (secreto para firmar), Emisor, Audiencia y MinutosExpiracion. Se inyecta con IOptions<OpcionesJwt>.

ServicioToken.cs

Crea el JWT al iniciar sesión: arma los claims (id, nombre, correo, rol), los firma con la clave (HMAC-SHA256), le pone fecha de vencimiento y devuelve el texto del token. El cliente lo manda en cada petición: Authorization: Bearer <token>.

```
new Claim(ClaimTypes.Role, nombreRol)   // el rol viaja dentro del token
new JsonWebTokenHandler().CreateToken(descriptor);
```

Middleware (ManejadorExcepcionesMiddleware.cs)

Un middleware es código que se ejecuta en **cada petición**, antes o después del controlador (como una aduana). Este envuelve todo en try/catch: si un servicio lanza una excepción, la convierte en el código HTTP correcto (404, 409, 422, 401) con un JSON uniforme { estado, mensaje }. Si algo inesperado falla, responde 500 sin mostrar detalles técnicos.

```
catch (ExcepcionNoEncontrada) -> 404      catch (ExcepcionConflicto) -> 409
catch (ExcepcionReglaNegocio) -> 422      catch (Exception) -> 500
```

Inicialización / SembradorDatos.cs

'Seed' = datos iniciales. Al arrancar, crea los 3 roles, los usuarios de prueba, especies y razas, vacunas, medicamentos, productos y veterinarios, y (si se activa) clientes y mascotas de ejemplo. Solo inserta si la tabla está vacía, así nunca duplica.

Program.cs

Es el punto de entrada. En orden: (1) lee la configuración, (2) registra servicios con AddScoped, (3) configura JWT y Swagger, (4) crea la base con EnsureCreated y siembra datos, (5) arma el pipeline: middleware de errores -> archivos del frontend -> Swagger -> autenticación -> autorización -> controladores.

Autenticación vs autorización

Autenticar = quién eres (login, token válido, 401 si falla). **Autorizar** = qué puedes hacer (rol, 403 si no tienes permiso). Una política global exige token en TODOS los endpoints salvo los marcados [AllowAnonymous] (login y registro).

wwwroot (frontend)

index.html (acceso), panel.html (panel), css/estilos.css, js/api.js (fetch con el token y manejo de errores), js/panel.js (clientes, mascotas, memorial) y js/modulos.js (el resto de pantallas, definidas con objetos de configuración).

6. Pruebas unitarias (VetCare.Pruebas)

xUnit + Moq (mocks)

Cada prueba crea el servicio con repositorios **falsos** (mocks): no hace falta base de datos. **Setup** define qué devuelve el falso, **Verify** comprueba que se llamó (o no) a un método, y **Assert** compara el resultado. [Fact] = una prueba; [Theory] + [InlineData] = la misma prueba con varios datos.

```
_clientes.Setup(c => c.BuscarPorDocumentoAsync("123")).ReturnsAsync(existente);
await Assert.ThrowsAsync<ExcepcionConflicto>(() => _servicio.CrearAsync(dto));
_clientes.Verify(c => c.CrearAsync(It.IsAny<Cliente>()), Times.Never);
```

7. Frases para la exposición

Respuestas cortas

¿Por qué capas? Cada capa tiene una sola responsabilidad: se prueba, se cambia y se entiende por separado. **¿Por qué ORM?** Evita SQL manual y errores, y el código no depende del motor de base de datos. **¿Por qué JWT?** El servidor no guarda sesiones: el token lleva el rol y se valida por firma. **¿Por qué DTOs?** Para no exponer las entidades ni el hash. **¿Por qué mocks?** Para probar la lógica sin depender de la base de datos.